

REGRESSION ANALYSIS

As the “Hello World” of machine learning algorithms, regression analysis is a simple supervised learning technique used to find the best trendline to describe a dataset.

The first regression analysis technique that we will examine is linear regression, which uses a straight line to describe a dataset. To unpack this simple technique, let’s return to the earlier dataset charting Bitcoin values to the US Dollar.

Date	Bitcoin Price	No. of Days Transpired
19-05-2015	234.31	1
14-01-2016	431.76	240
09-07-2016	652.14	417
15-01-2017	817.26	607
24-05-2017	2358.96	736

Imagine you’re back in high school and it's the year 2015 (which is probably much more recent than your actual year of graduation!). During your senior year, a news headline piques your interest in Bitcoin. With your natural tendency to chase the next shiny object, you tell your family about your cryptocurrency aspirations. But before you have a chance to bid for your first Bitcoin on Coinbase, your father intervenes and insists that you try paper trading before you go risking your life savings. “Paper trading” is using simulated means to buy and sell an investment without involving actual money.

So over the next twenty-four months, you track the value of Bitcoin and write down its value at regular intervals. You also keep a tally of how many days have passed since you first started paper trading. You never anticipated to still be paper trading after two years, but unfortunately, you never got a

chance to enter the cryptocurrency market. As suggested by your father, you waited for the value of Bitcoin to drop to a level you could afford. But instead, the value of Bitcoin exploded in the opposite direction.

Nonetheless, you haven't lost hope of one day owning Bitcoin. To assist your decision on whether you continue to wait for the value to drop or to find an alternative investment class, you turn your attention to statistical analysis. You first reach into your toolbox for a scatterplot. With the blank scatterplot in your hands, you proceed to plug in your x and y coordinates from your dataset and plot Bitcoin values from 2015 to 2017. However, rather than use all three columns from the table, you select the second (Bitcoin price) and third (No. of Days Transpired) columns to build your model and populate the scatterplot (shown in Figure 1). As we know, numerical values (found in the second and third columns) are easy to plug into a scatterplot and require no special conversion or one-hot encoding. What's more, the first and third columns contain the same variable of "time" and the third column alone is sufficient.

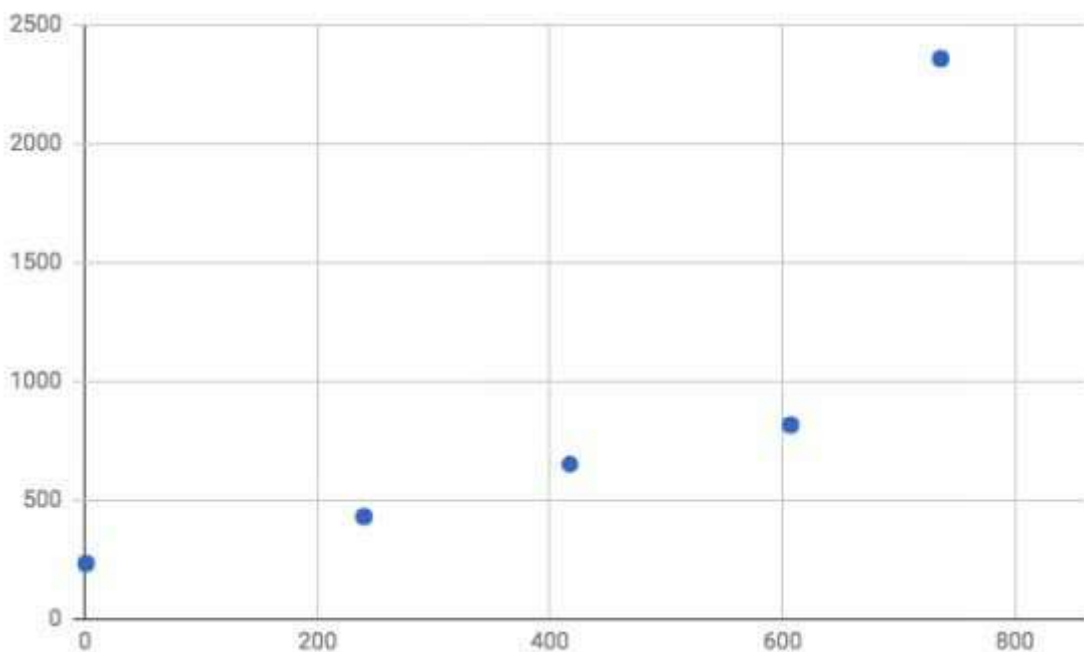


Figure 1: Bitcoin values from 2015-2017 plotted on a scatterplot

As your goal is to estimate what Bitcoin will be valued at in the future, the y-axis plots the dependent variable, which is "Bitcoin Price." The independent variable (X), in this case, is time. The "No. of Days Transpired" is thereby plotted on the x-axis.

After plotting the x and y values on the scatterplot, you can immediately see a trend in the form of a curve ascending from left to right with a steep increase between day 607 and day 736. Based on the upward trajectory of the curve, it might be time to quit hoping for a drop in value.

However, an idea suddenly pops up into your head. What if instead of waiting for the value of Bitcoin to fall to a level that you can afford, you instead borrow from a friend and purchase Bitcoin now at day 736? Then, when the value of Bitcoin rises further, you can pay back your friend and continue to earn asset appreciation on the Bitcoin you fully own.

In order to assess whether it's worth borrowing from your friend, you will need to first estimate how much you can earn in potential profit. Then you need to figure out whether the return on investment will be adequate to pay back your friend in the short-term.

It's now time to reach into the third compartment of the toolbox for an algorithm. One of the simplest algorithms in machine learning is regression analysis, which is used to determine the strength of a relationship between variables. Regression analysis comes in many forms, including linear, non-linear, logistic, and multilinear, but let's take a look first at linear regression.

Linear regression comprises a straight line that splits your data points on a scatterplot. The goal of linear regression is to split your data in a way that minimizes the distance between the regression line and all data points on the scatterplot. This means that if you were to draw a vertical line from the regression line to each data point on the graph, the aggregate distance of each point would equate to the smallest possible distance to the regression line.

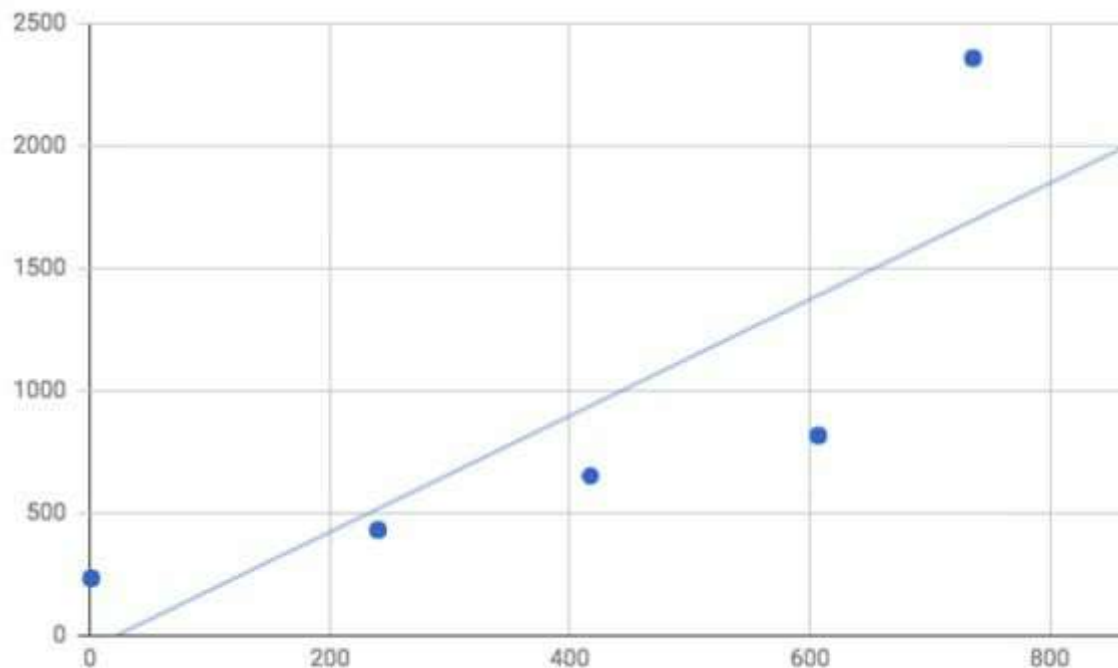


Figure 2: Linear regression line

The regression line is plotted on the scatterplot in Figure 2. The technical term for the regression line is the *hyperplane*, and you will see this term used throughout your study of machine learning. A hyperplane is practically a trendline—and this is precisely how Google Sheets titles linear regression in its scatterplot customization menu.

Another important feature of regression is *slope*, which can be conveniently calculated by referencing the hyperplane. As one variable increases, the other variable will increase at the average value denoted by the hyperplane. The slope is therefore very useful in formulating predictions. For example, if you wish to estimate the value of Bitcoin at 800 days, you can enter 800 as your x coordinate and reference the slope by finding the corresponding y value represented on the hyperplane. In this case, the y value is USD \$1,850.

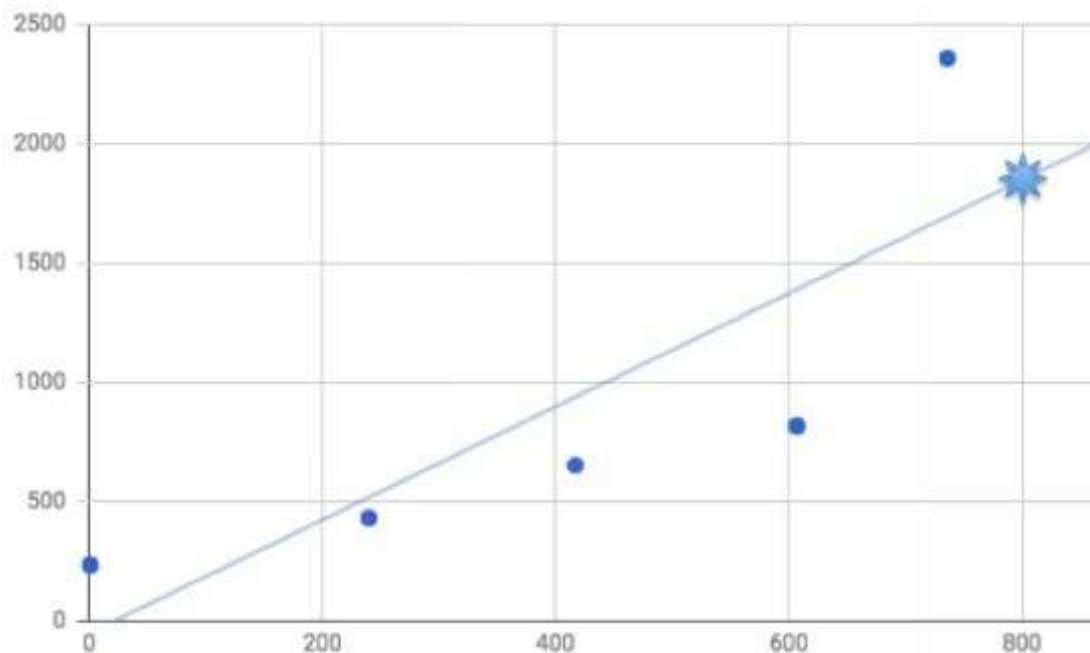


Figure 3: The value of Bitcoin at day 800

As shown in Figure 3, the hyperplane reveals that you actually stand to lose money on your investment at day 800 (after buying on day 736)! Based on the slope of the hyperplane, Bitcoin is expected to depreciate in value between day 736 and day 800—despite no precedent in your dataset for Bitcoin ever dropping in value.

While it's needless to say that linear regression isn't a fail-proof method to picking investment trends, the trendline does offer a basic reference point to predict the future. If we were to use the trendline as a reference point earlier in time, say at day 240, then the prediction posted would have been more accurate. At day 240 there is a low degree of deviation from the hyperplane, while at day 736 there is a high degree of deviation. Deviation refers to the distance between the hyperplane and the data point.

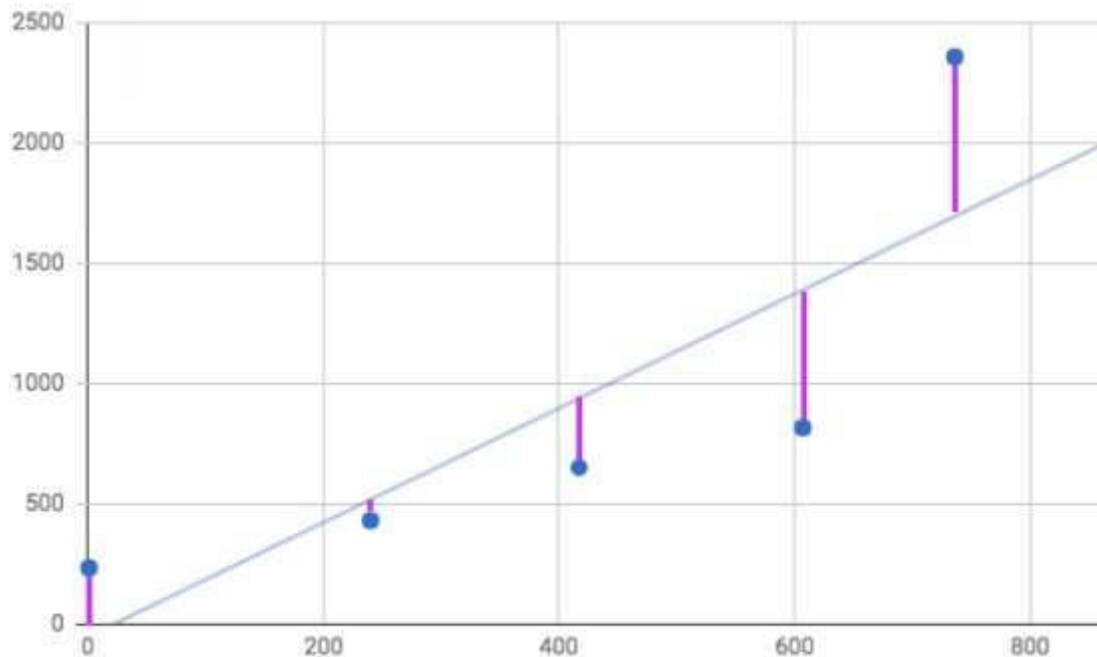


Figure 4: The distance of the data points to the hyperplane

In general, the closer the data points are to the regression line, the more accurate the final prediction. If there is a high degree of deviation between the data points and the regression line, the slope will provide less accurate predictions. Basing your predictions on the data point at day 736, where there is high deviation, results in poor accuracy. In fact, the data point at day 736 constitutes an outlier because it does not follow the same general trend as the previous four data points. What's more, as an outlier it exaggerates the trajectory of the hyperplane based on its high y-axis value. Unless future data points scale in proportion to the y-axis values of the outlier data point, the model's predictive accuracy will suffer.

Calculation Example

Although your programming language will take care of this automatically, it's useful to understand how linear regression is actually calculated. We will use the following dataset and formula to perform linear regression.

	(X)	(Y)	XY	X ²
1	1	3	3	1
2	2	4	8	4
3	1	2	2	1
4	4	7	28	16
5	3	5	15	9
Σ (Total)	11	21	56	31

The final two columns of the table are not part of the original dataset and have been added for convenience to complete the following equation.

$$a = \frac{(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)}{n(\Sigma x^2) - (\Sigma x)^2}$$

$$b = \frac{n(\Sigma xy) - (\Sigma x)(\Sigma y)}{n(\Sigma x^2) - (\Sigma x)^2}$$

Where:

Σ = Total sum

Σx = Total sum of all x values (1 + 2 + 1 + 4 + 3 = 11)

Σy = Total sum of all y values (3 + 4 + 2 + 7 + 5 = 21)

Σxy = Total sum of x*y for each row (3 + 8 + 2 + 28 + 15 = 56)

Σx² = Total sum of x*x for each row (1 + 4 + 1 + 16 + 9 = 31)

n = Total number of rows. In the case of this example, n = 5.

$$a = \frac{(\sum y)(\sum x^2) - (\sum x)(\sum xy)}{n(\sum x^2) - (\sum x)^2}$$

$$a = \frac{(21)(31) - (11)(56)}{5(31) - (11)^2}$$

$$b = \frac{n(\sum xy) - (\sum x)(\sum y)}{n(\sum x^2) - (\sum x)^2}$$

$$b = \frac{5(56) - (11)(21)}{5(31) - (11)^2}$$

A =

$$((21 \times 31) - (11 \times 56)) / (5(31) - 11^2)$$

$$(651 - 616) / (155 - 121)$$

$$35 / 34$$

$$1.029$$

B =

$$(5(56) - (11 \times 21)) / (5(31) - 11^2)$$

$$(280 - 231) / (155 - 121)$$

$$49 / 34$$

$$1.44$$

Insert the “a” and “b” values into a linear equation.

$$y = a + bx$$

$$y = 1.029 + 1.441x$$

The linear equation $y = 1.029 + 1.441x$ dictates how to draw the hyperplane.

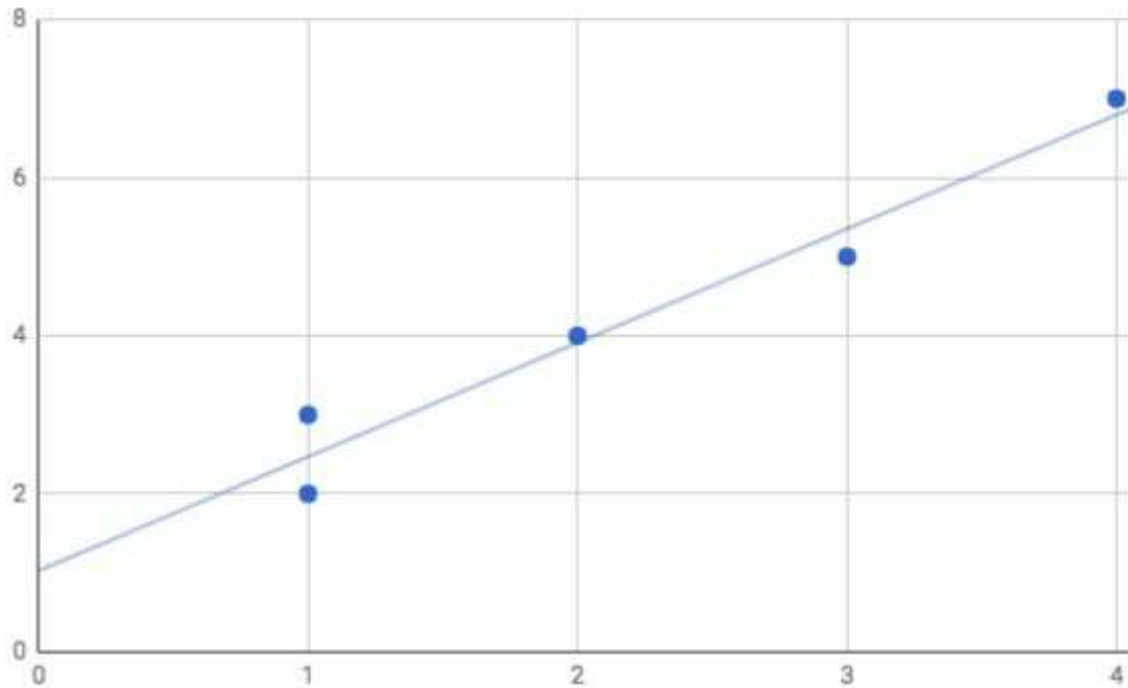


Figure 5: The linear regression hyperplane plotted on the scatterplot

Let's now test the regression line by looking up the coordinates for $x = 2$.

$$y = 1.029 + 1.441(x)$$

$$y = 1.029 + 1.441(2)$$

$$y = 3.911$$

In this case, the prediction is very close to the actual result of 4.0.

Logistic Regression

A large part of data analysis boils down to a simple question: is something “A” or “B?” Is it “positive” or “negative?” Is this person a “potential customer” or “not a potential customer?” Machine learning accommodates such questions through logistic equations, and specifically through what is known as the *sigmoid function*. The sigmoid function produces an S-shaped curve that can convert any number and map it into a numerical value between 0 and 1, but it does so without ever reaching those exact limits.

A common application of the sigmoid function is found in logistic regression. Logistic regression adopts the sigmoid function to analyze data and predict discrete classes that exist in a dataset. Although logistic regression shares a visual resemblance to linear regression, it is technically a classification technique. Whereas linear regression addresses numerical equations and forms numerical predictions to discern relationships between variables,

logistic regression predicts discrete classes.

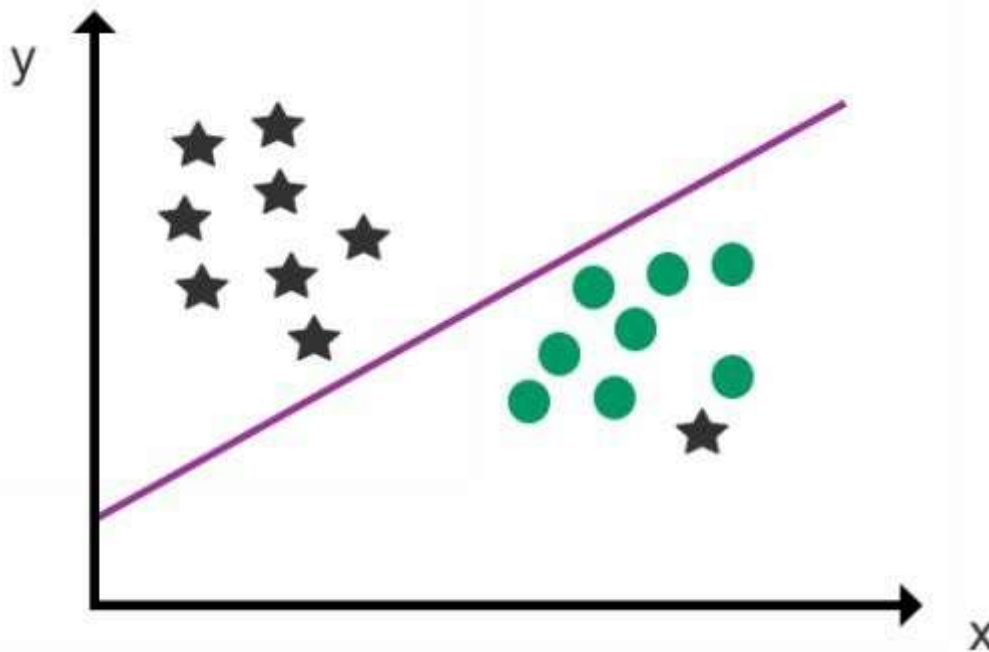


Figure 6: An example of logistic regression

Logistic regression is typically used for binary classification to predict two discrete classes, e.g. *pregnant* or *not pregnant*. To do this, the sigmoid function (shown as follows) is added to compute the result and convert numerical results into an expression of probability between 0 and 1.

$$y = \frac{1}{1+e^{-x}}$$

The logistic sigmoid function above is calculated as “1” divided by “1” plus “e” raised to the power of negative “x,” where:

x = the numerical value you wish to transform

e = Euler's constant, 2.718

In a binary case, a value of 0 represents no chance of occurring, and 1 represents a certain chance of occurring. The degree of probability for values located between 0 and 1 can be calculated according to how close they rest to 0 (impossible) or 1 (certain possibility) on the scatterplot.

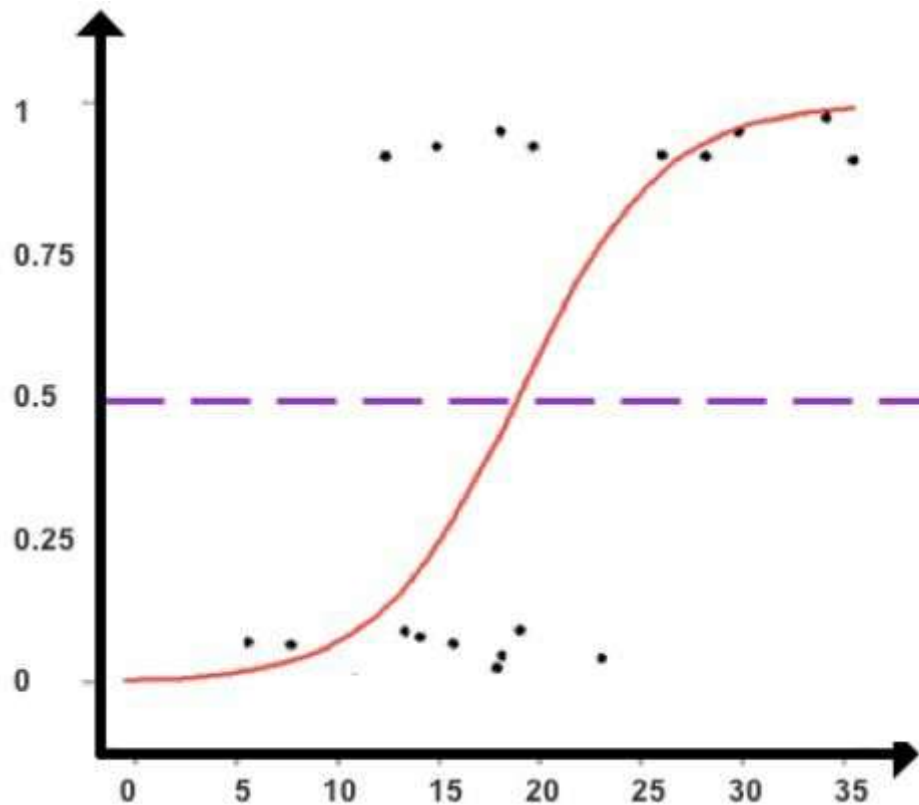


Figure 7: A sigmoid function used to classify data points

Based on the found probabilities we can assign each data point to one of two discrete classes. As seen in Figure 7, we can create a cut-off point at 0.5 to classify the data points into classes. Data points that record a value above 0.5 are classified as Class A, and any data points below 0.5 are classified as Class B. Data points that record a result of exactly 0.5 are unclassifiable, but such instances are rare due to the mathematical component of the sigmoid function.

Please also note that this formula alone does not produce the hyperplane dividing discrete categories as seen earlier in Figure 6. The statistical formula for plotting the logistic hyperplane is somewhat more complicated and can be conveniently plotted using your programming language.

Given its strength in binary classification, logistic regression is used in many fields including fraud detection, disease diagnosis, emergency detection, loan default detection, or to identify spam email through the process of identifying specific classes, e.g. non-spam and spam. However, logistic regression can also be applied to ordinal cases where there are a set number of discrete values, e.g. single, married, and divorced.

Logistic regression with more than two outcome values is known as

multinomial logistic regression, which can be seen in Figure 8.

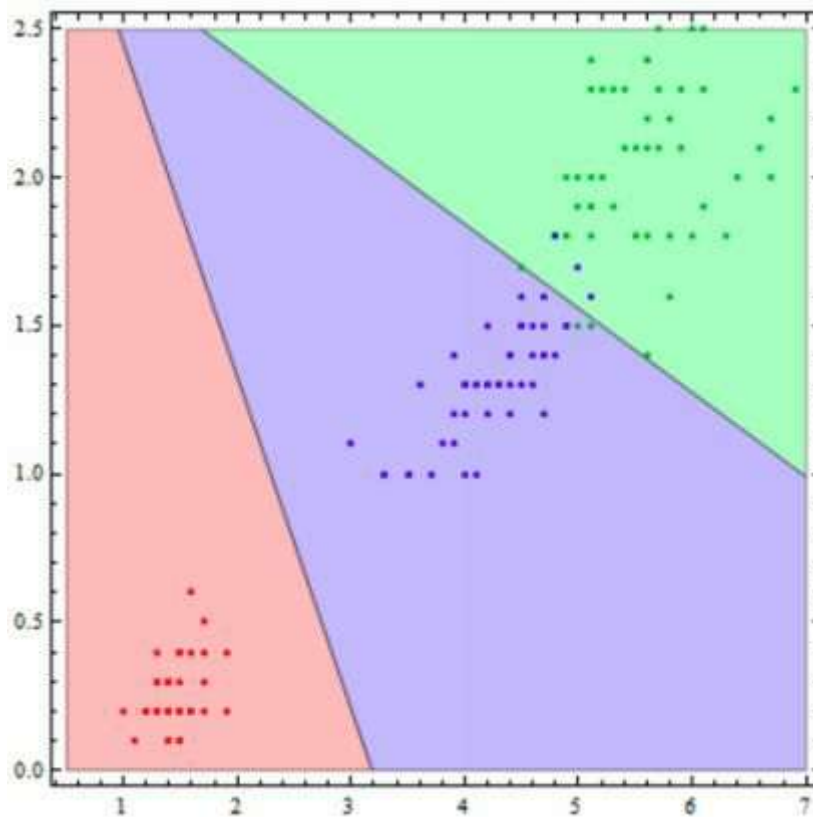


Figure 8: An example of multinomial logistic regression

Two tips to remember when performing logistic regression are that the data should be free of missing values and that all variables are independent of each other. There should also be sufficient data for each outcome value to ensure high accuracy. A good starting point would be approximately 30-50 data points for each outcome, i.e. 60-100 total data points for binary logistic regression.

Support Vector Machine

As an advanced category of regression, support vector machine (SVM) resembles logistic regression but with stricter conditions. To that end, SVM is superior at drawing classification boundary lines. Let's examine what this looks like in action.

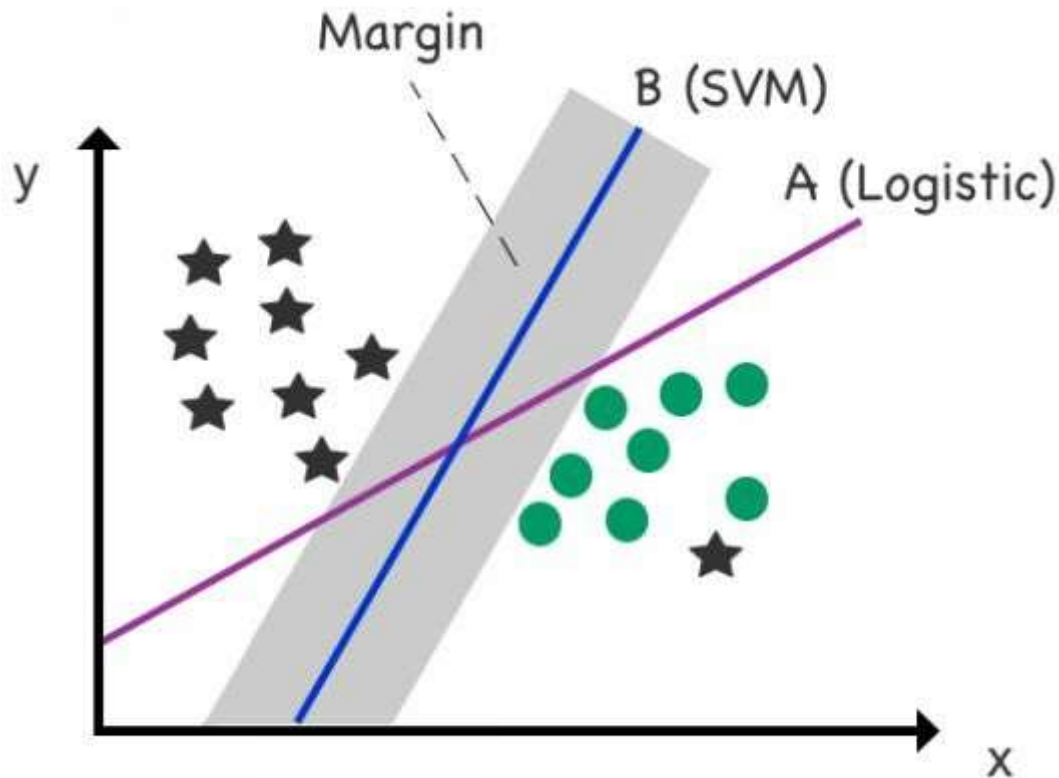


Figure 9: Logistic regression versus SVM

The scatterplot in Figure 9 consists of data points that are linearly separable and the logistic hyperplane (A) splits the data points into two classes in a way that minimizes the distance between all data points and the hyperplane. The second line, the SVM hyperplane (B), likewise separates the two clusters, but from a position of maximum distance between itself and the two clusters.

You will also notice a gray area that denotes *margin*, which is the distance between the hyperplane and the nearest data point, multiplied by two. The margin is a key feature of SVM and is important because it offers additional support to cope with new data points that may infringe on a logistic regression hyperplane. To illustrate this scenario, let's consider the same scatterplot with the inclusion of a new data point.

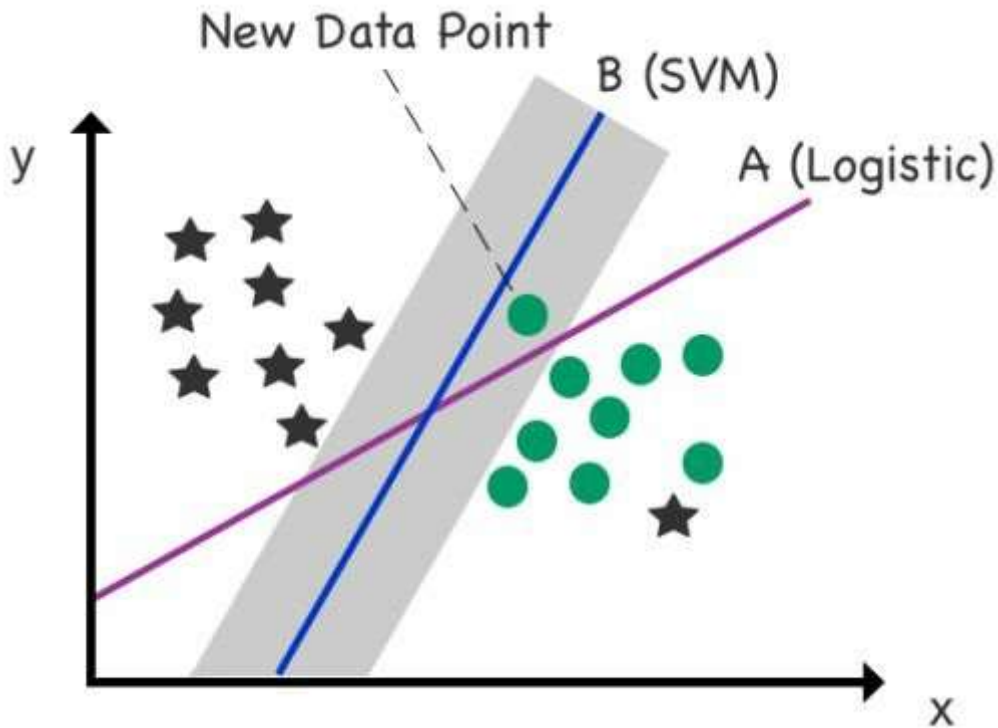


Figure 10: A new data point is added to the scatterplot

The new data point is a circle, but it is located incorrectly on the left side of the logistic regression hyperplane (designated for stars). The new data point, though, remains correctly located on the right side of the SVM hyperplane (designated for circles) courtesy of ample “support” supplied by the margin.

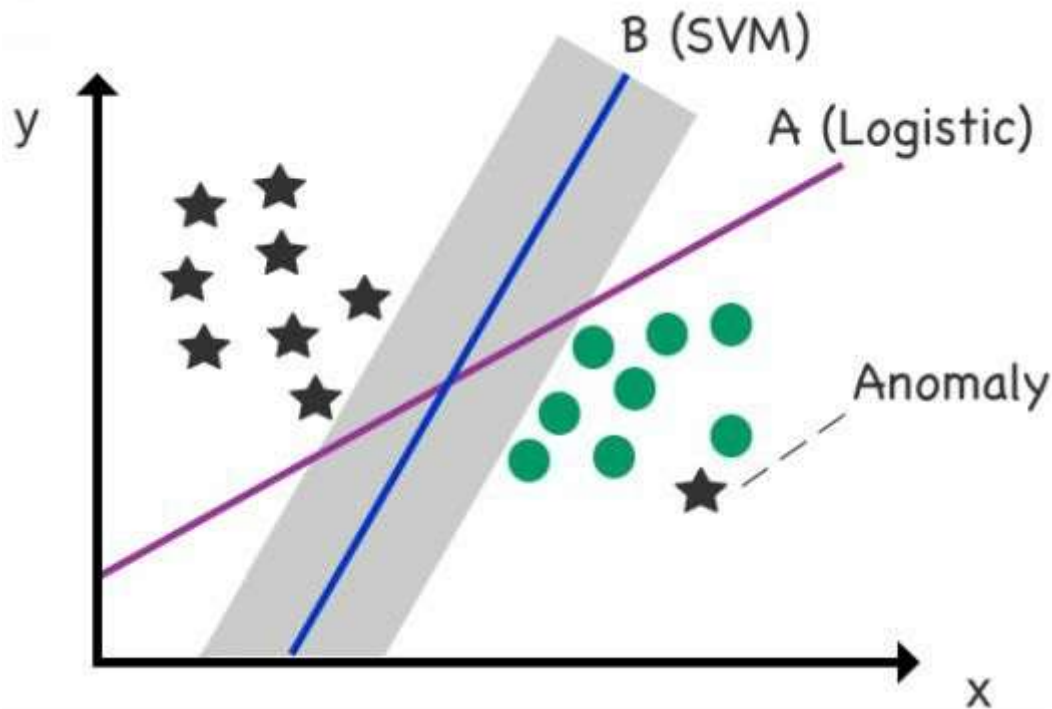


Figure 11: Mitigating anomalies

Another useful application case of SVM is for mitigating anomalies. A limitation of standard logistic regression is that it goes out of its way to fit anomalies (as seen in the scatterplot with the star in the bottom right corner in Figure 11). SVM, however, is less sensitive to such data points and actually minimizes their impact on the final location of the boundary line. In Figure 11, we can see that Line B (SVM hyperplane) is less sensitive to the anomalous star on the right-hand side. SVM can thus be used as one method to fight anomalies.

The examples seen so far have comprised two features plotted on a two-dimensional scatterplot. However, SVM's real strength is found in high-dimensional data and handling multiple features. SVM has numerous variations available to classify high-dimensional data, known as "kernels," including linear SVC (seen in Figure 12), polynomial SVC, and the Kernel Trick. The Kernel Trick is an advanced solution to map data from a low-dimensional to a high-dimensional space. Transitioning from a two-dimensional to a three-dimensional space allows you to use a linear plane to split the data within a 3-D space, as seen in Figure 12.

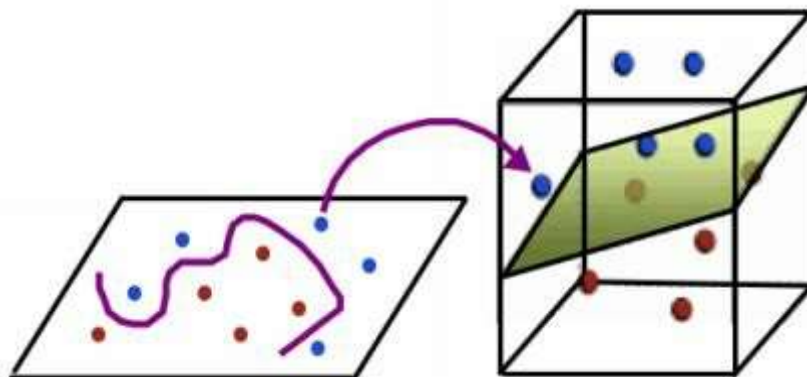


Figure 12: Example of linear SVC

CLUSTERING

One helpful approach to analyze information is to identify clusters of data that share similar attributes. For example, your company may wish to examine a segment of customers that purchase at the same time of the year and discern what factors influence their purchasing behavior.

By understanding a particular cluster of customers, you can form decisions about which products to recommend to customer groups through promotions and personalized offers. Outside of market research, clustering can be applied to various other scenarios, including pattern recognition, fraud detection, and image processing.

Clustering analysis falls under the banner of both supervised learning and unsupervised learning. As a supervised learning technique, clustering is used to classify new data points into existing clusters through k -nearest neighbors (k -NN) and as an unsupervised learning technique, clustering is applied to identify discrete groups of data points through k -means clustering. Although there are other forms of clustering techniques, these two algorithms are generally the most popular in both machine learning and data mining.

k -Nearest Neighbors

The simplest clustering algorithm is k -nearest neighbors (k -NN); a supervised learning technique used to classify new data points based on the relationship to nearby data points.

k -NN is similar to a voting system or a popularity contest. Think of it as being the new kid in school and choosing a group of classmates to socialize with based on the five classmates who sit nearest to you. Among the five classmates, three are **geeks**, one is a **skater**, and one is a **jock**. According to k -NN, you would choose to hang out with the **geeks** based on their numerical advantage. Let's look at another example.

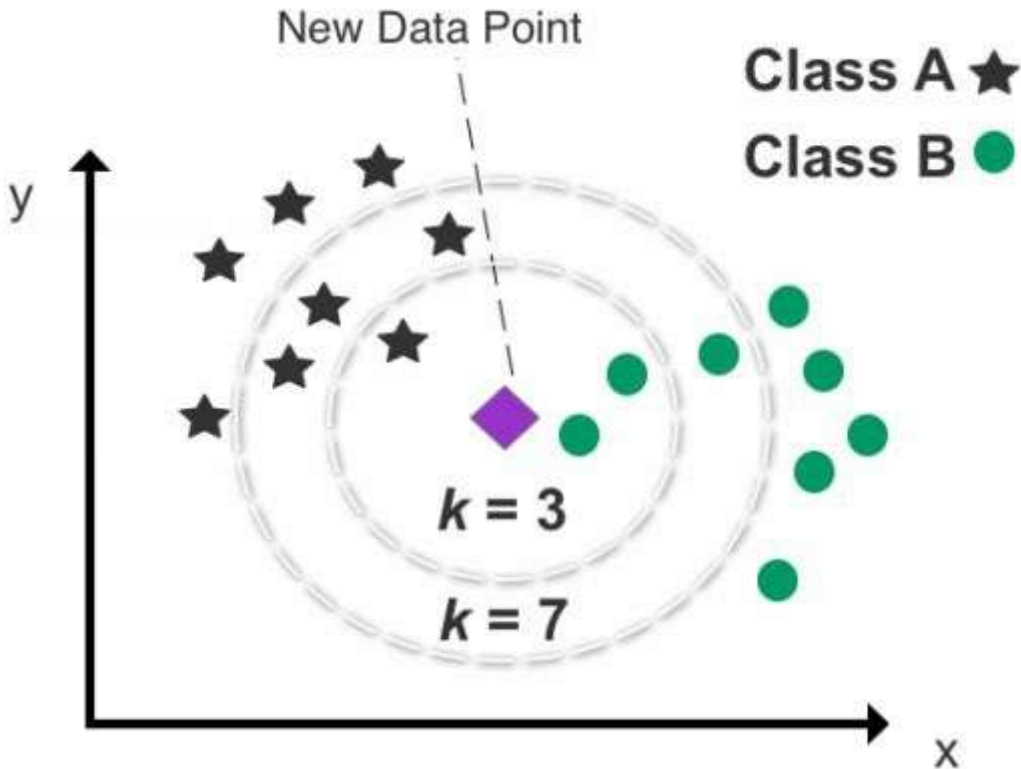


Figure 1: An example of k -NN clustering used to predict the class of a new data point

As seen in Figure 1, the scatterplot enables us to compute the distance between any two data points. The data points on the scatterplot have already been categorized into two clusters. Next, a new data point whose class is unknown is added to the plot. We can predict the category of the new data point based on its relationship to existing data points.

First though, we must set “ k ” to determine how many data points we wish to nominate to classify the new data point. If we set k to 3, k -NN will only analyze the new data point’s relationship to the three closest data points (neighbors). The outcome of selecting the three closest neighbors returns two Class B data points and one Class A data point. Defined by k (3), the model’s prediction for determining the category of the new data point is Class B as it returns two out of the three nearest neighbors.

The chosen number of neighbors identified, defined by k , is crucial in determining the results. In Figure 1, you can see that classification will change depending on whether k is set to “3” or “7.” It is therefore recommended that you test numerous k combinations to find the best fit and avoid setting k too low or too high. Setting k to an uneven number will also help to eliminate the possibility of a statistical stalemate and invalid result. The default number of neighbors is five when using Scikit-learn.

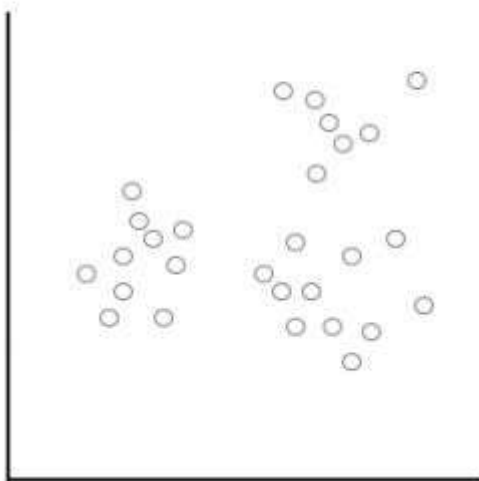
Although generally a highly accurate and simple technique to learn, storing an entire dataset and calculating the distance between each new data point and all existing data points does place a heavy burden on computing resources. Thus, k -NN is generally not recommended for use with large datasets.

Another potential downside is that it can be challenging to apply k -NN to high-dimensional data (3-D and 4-D) with multiple features. Measuring multiple distances between data points in a three or four-dimensional space is taxing on computing resources and also complicated to perform accurate classification. Reducing the total number of dimensions, through a descending dimension algorithm such as Principle Component Analysis (PCA) or merging variables, is a common strategy to simplify and prepare a dataset for k -NN analysis.

k -Means Clustering

As a popular unsupervised learning algorithm, k -means clustering attempts to divide data into k discrete groups and is effective at uncovering basic data patterns. Examples of potential groupings include animal species, customers with similar features, and housing market segmentation. The k -means clustering algorithm works by first splitting data into k number of clusters with k representing the number of clusters you wish to create. If you choose to split your dataset into three clusters then k , for example, is set to 3.

Original Data



Clustered Data

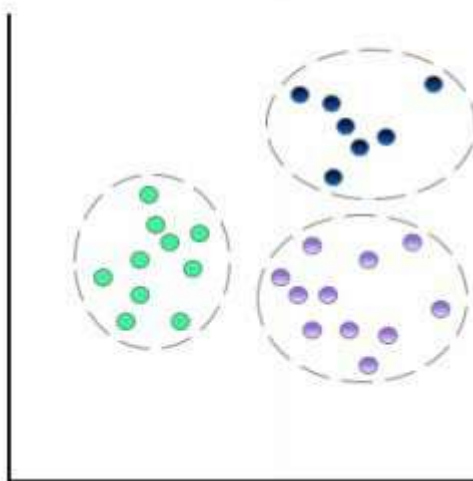


Figure 2: Comparison of original data and clustered data using k -means

In Figure 2, we can see that the original (unclustered) data has been transformed into three clusters (k is 3). If we were to set k to 4, an additional cluster would be derived from the dataset to produce four clusters.

How does k -means clustering separate the data points? The first step is to examine the unclustered data on the scatterplot and manually select a centroid for each k cluster. That centroid then forms the epicenter of an individual cluster. Centroids can be chosen at random, which means you can nominate any data point on the scatterplot to act as a centroid. However, you can save time by choosing centroids dispersed across the scatterplot and not directly adjacent to each other. In other words, start by guessing where you think the centroids for each cluster might be located. The remaining data points on the scatterplot are then assigned to the closest centroid by measuring the Euclidean distance.

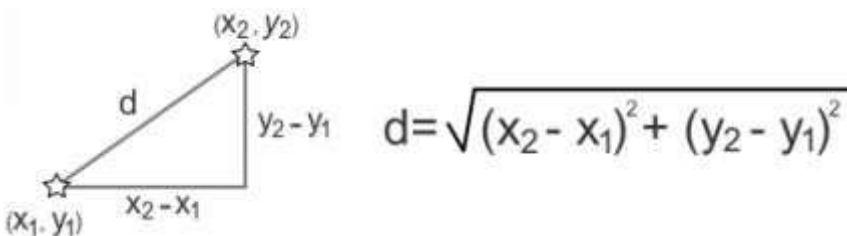


Figure 3: Calculating Euclidean distance

Each data point can be assigned to only one cluster and each cluster is discrete. This means that there is no overlap between clusters and no case of nesting a cluster inside another cluster. Also, all data points, including anomalies, are assigned to a centroid irrespective of how they impact the final shape of the cluster. However, due to the statistical force that pulls all nearby data points to a central point, your clusters will generally form an elliptical or spherical shape.

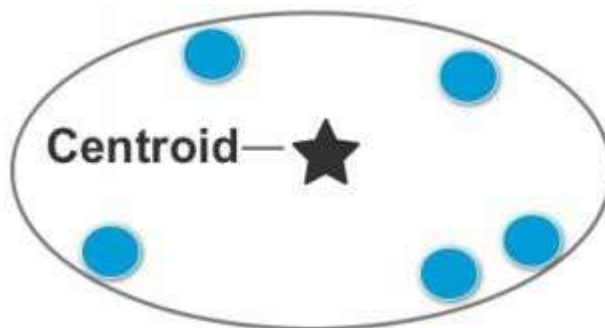


Figure 4: Example of an ellipse cluster

After all data points have been allocated to a centroid, the next step is to aggregate the mean value of all data points for each cluster, which can be found by calculating the average x and y values of all data points in that cluster.

Next, take the mean value of the data points in each cluster and plug in those x and y values to update your centroid coordinates. This will most likely result in a change to your centroids' location. Your total number of clusters, however, will remain the same. You are not creating new clusters, rather updating their position on the scatterplot. Like musical chairs, the remaining data points will then rush to the closest centroid to form k number of clusters. Should any data point on the scatterplot switch clusters with the changing of centroids, the previous step is repeated. This means, again, calculating the average mean value of the cluster and updating the x and y values of each centroid to reflect the average coordinates of the data points in that cluster.

Once you reach a stage where the data points no longer switch clusters after an update in centroid coordinates, the algorithm is complete, and you have your final set of clusters. The following diagrams break down the full algorithmic process.

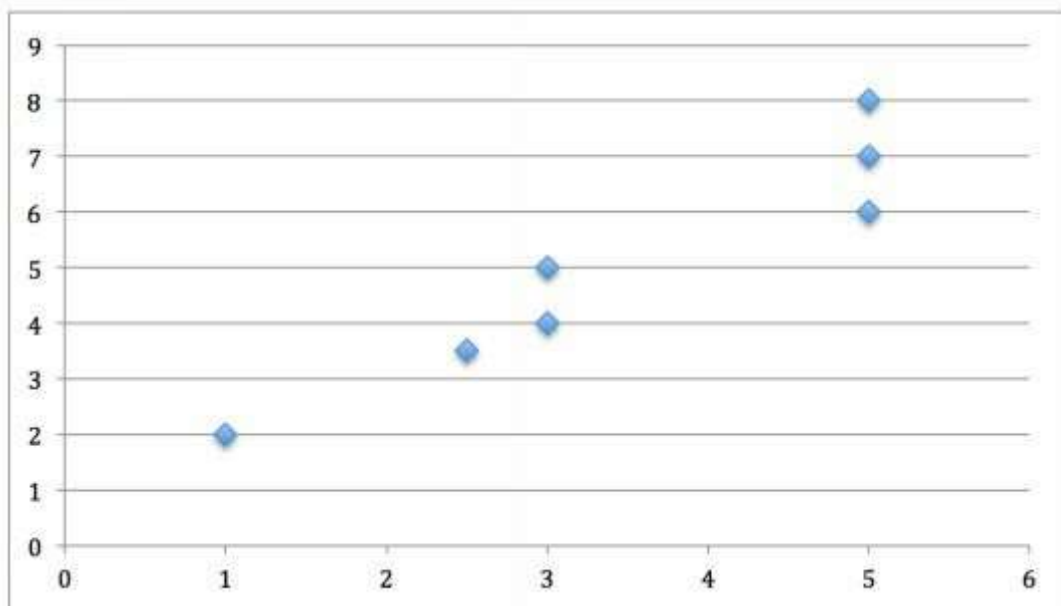


Figure 5: Sample data points are plotted on a scatterplot

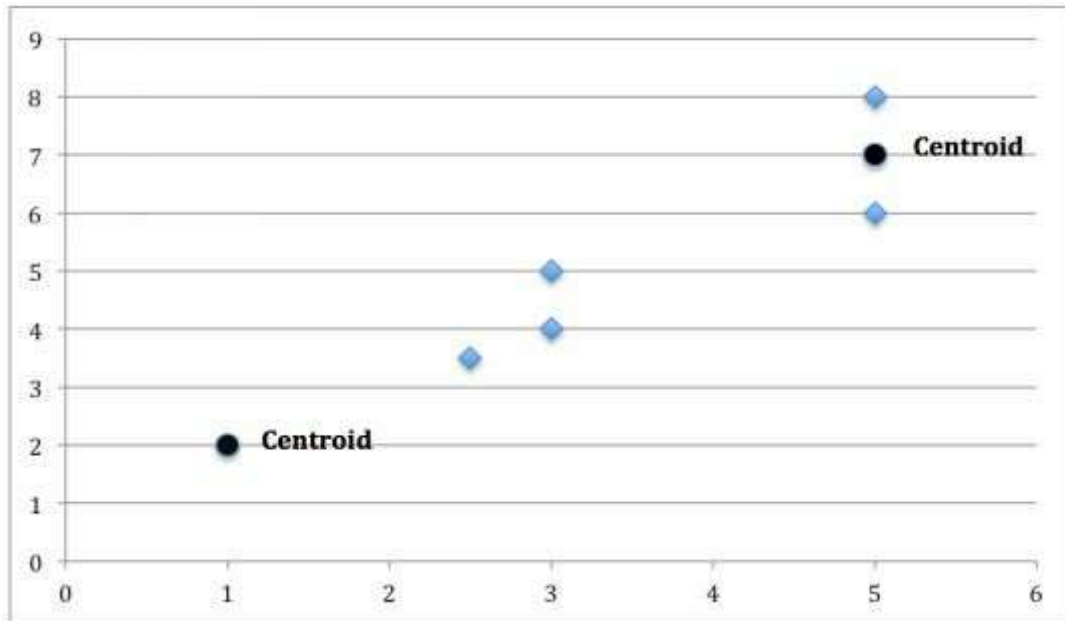


Figure 6: Two data points are nominated as centroids

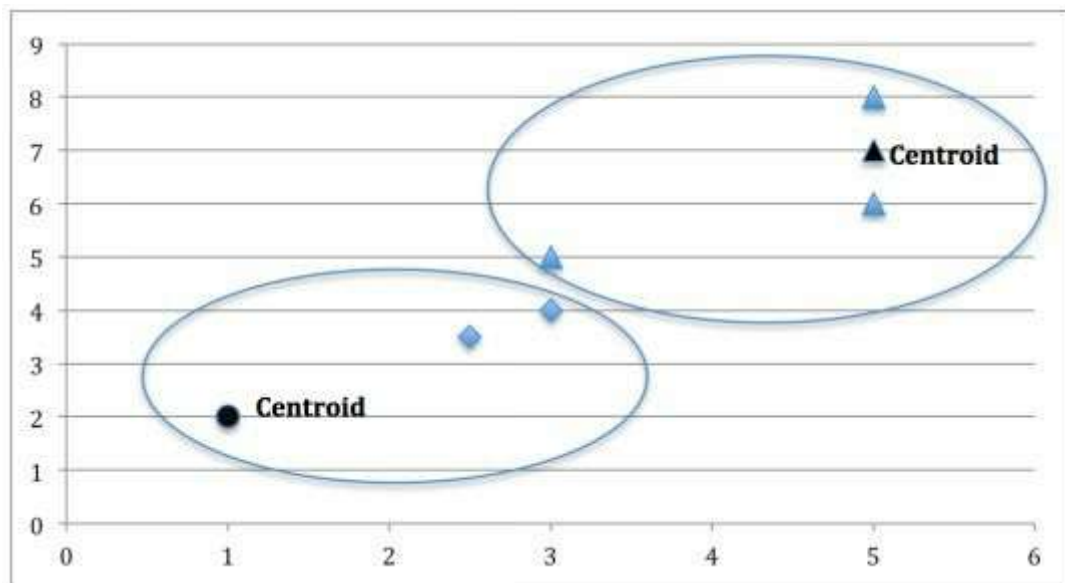


Figure 7: Two clusters are formed after calculating the Euclidean distance of the remaining data points to the centroids.

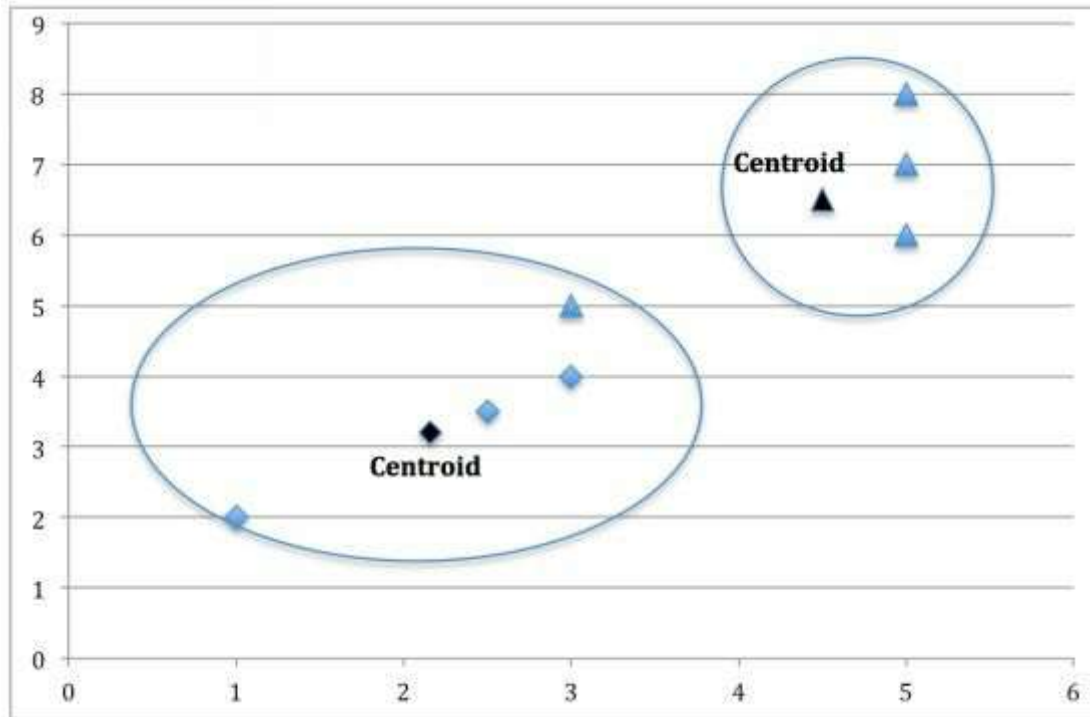


Figure 8: The centroid coordinates for each cluster are updated to reflect the cluster's mean value. As one data point has switched from the right cluster to the left cluster, the centroids of both clusters are recalculated.

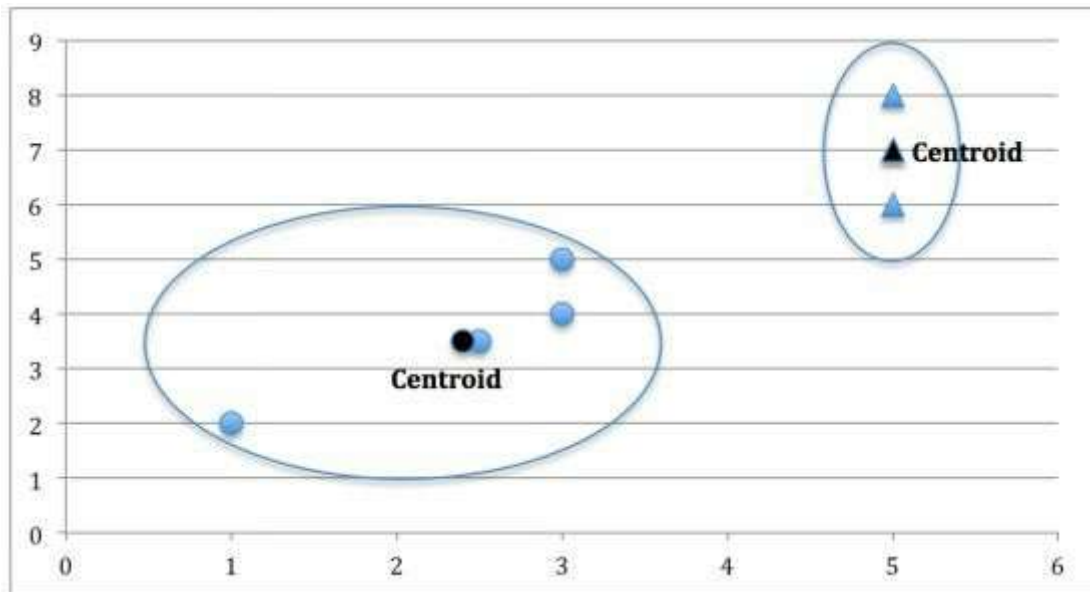


Figure 9: Two final clusters are produced based on the updated centroids for each cluster

Setting k

In setting k , it is important to strike the right number of clusters. In general, as k increases, clusters become smaller and variance falls. However, the downside is that neighboring clusters become less distinct from one another as k increases.

If you set k to the same number of data points in your dataset, each data point automatically converts into a standalone cluster. Conversely, if you set k to 1, then all data points will be deemed as homogenous and produce only one cluster. Needless to say, setting k to either extreme will not provide any worthy insight to analyze.

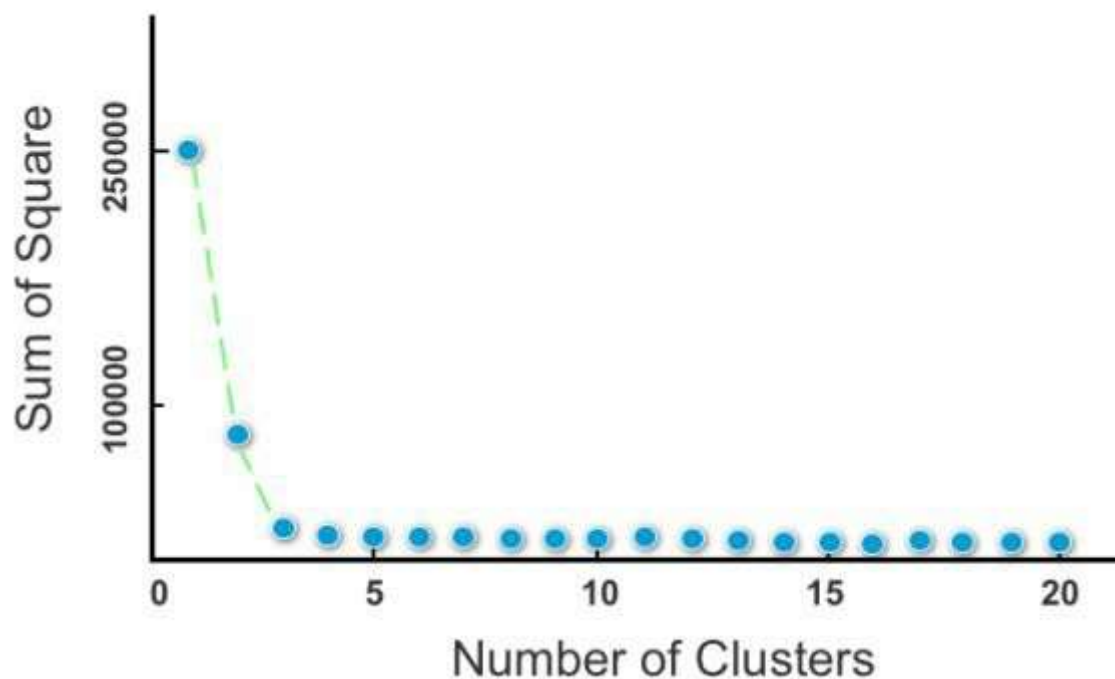


Figure 10: A scree plot

In order to optimize k , you may wish to turn to a scree plot for guidance. A scree plot charts the degree of scattering (variance) inside a cluster as the total number of clusters increase. Scree plots are famous for their iconic “elbow,” which reflects several pronounced kinks in the plot’s curve.

A scree plot compares the Sum of Squared Error (SSE) for each variation of total clusters. SSE is measured as the sum of the squared distance between the centroid and the other neighbors inside the cluster. In a nutshell, SSE drops as more clusters are formed.

This then raises the question of what the optimal number of clusters is. In general, you should opt for a cluster solution where SSE subsides dramatically to the left on the scree plot, but before it reaches a point of negligible change with cluster variations to its right. For instance, in Figure 10, there is little impact on SSE for six or more clusters. This would result in clusters that would be small and difficult to distinguish.

In this scree plot, two or three clusters appear to be an ideal solution. There

exists a significant kink to the left of these two cluster variations due to a pronounced drop-off in SSE. Meanwhile, there is still some change in SSE with the solution to their right. This will ensure that these two cluster solutions are distinct and have an impact on data classification.

A more simple and non-mathematical approach to setting k is applying domain knowledge. For example, if I am analyzing data concerning visitors to the website of a major IT provider, I might want to set k to 2. Why two clusters? Because I already know there is likely to be a major discrepancy in spending behavior between returning visitors and new visitors. First-time visitors rarely purchase enterprise-level IT products and services, as these customers will normally go through a lengthy research and vetting process before procurement can be approved.

Hence, I can use k -means clustering to create two clusters and test my hypothesis. After creating two clusters, I may then want to examine one of the two clusters further, either applying another technique or again using k -means clustering. For example, I might want to split returning users into two clusters (using k -means clustering) to test my hypothesis that mobile users and desktop users produce two disparate groups of data points. Again, by applying domain knowledge, I know it is uncommon for large enterprises to make big-ticket purchases on a mobile device. Still, I wish to create a machine learning model to test this assumption.

If, though, I am analyzing a product page for a low-cost item, such as a \$4.99 domain name, new visitors and returning visitors are less likely to produce two clear clusters. As the product item is of low value, new users are less likely to deliberate before purchasing.

Instead, I might choose to set k to 3 based on my three primary lead generators: organic traffic, paid traffic, and email marketing. These three lead sources are likely to produce three discrete clusters based on the facts that:

- a) **Organic traffic** generally consists of both new and returning customers with a strong intent of purchasing from my website (through pre-selection, e.g. word of mouth, previous customer experience).
- b) **Paid traffic** targets new customers who typically arrive on the website with a lower level of trust than organic traffic, including potential customers who click on the paid advertisement by mistake.
- c) **Email marketing** reaches existing customers who already have experience purchasing from the website and have established user accounts.

This is an example of domain knowledge based on my own occupation, but do understand that the effectiveness of “domain knowledge” diminishes dramatically past a low number of k clusters. In other words, domain knowledge might be sufficient for determining two to four clusters, but it will be less valuable in choosing between 20 or 21 clusters.

BIAS & VARIANCE

Algorithm selection is an important step in forming an accurate prediction model, but deploying an algorithm with a high rate of accuracy can be a difficult balancing act. The fact that each algorithm can produce vastly different models based on the hyperparameters provided can lead to dramatically different results. As mentioned earlier, hyperparameters are the algorithm's settings, similar to the controls on the dashboard of an airplane or the knobs used to tune radio frequency—except hyperparameters are lines of code!

```
model = ensemble.GradientBoostingRegressor(  
    n_estimators=150,  
    learning_rate=0.1,  
    max_depth=4,  
    min_samples_split=4,  
    min_samples_leaf=4,  
    max_features=0.5,  
    loss='huber'  
)
```

Figure 1: Example of hyperparameters in Python for the algorithm gradient boosting

A constant challenge in machine learning is navigating *underfitting* and *overfitting*, which describe how closely your model follows the actual patterns of the dataset. To understand underfitting and overfitting, you must first understand *bias* and *variance*.

Bias refers to the gap between your predicted value and the actual value. In the case of high bias, your predictions are likely to be skewed in a certain direction away from the actual values. Variance describes how scattered your predicted values are. Bias and variance can be best understood by analyzing the following visual representation.

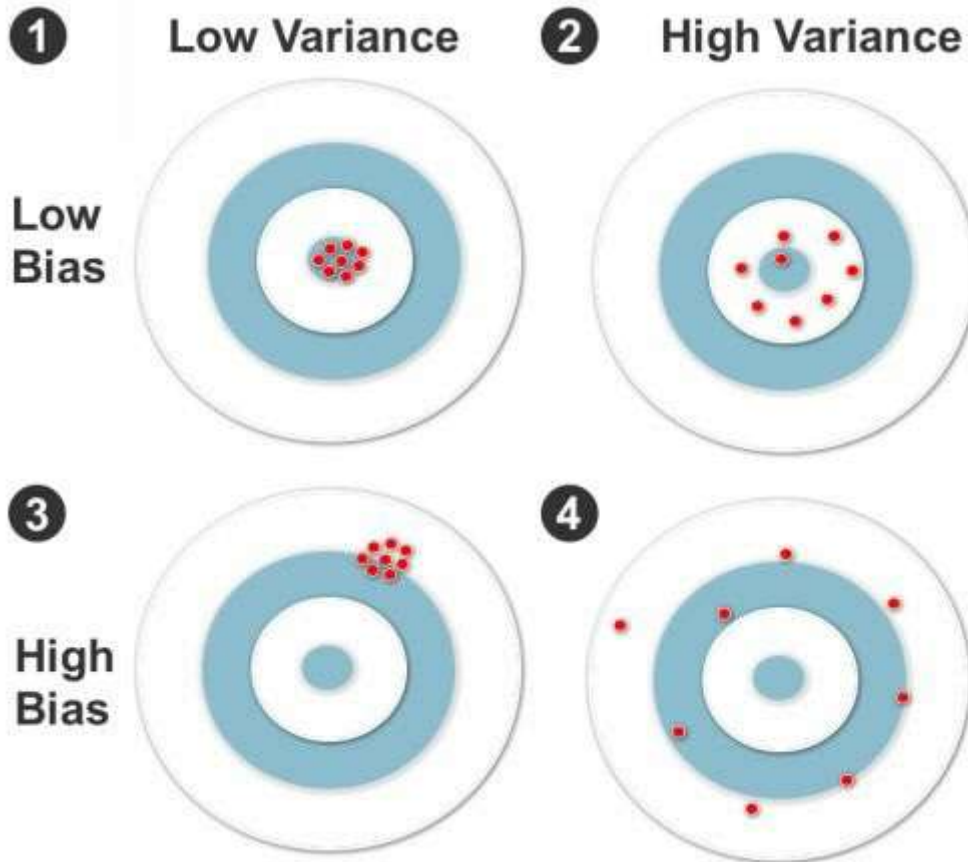


Figure 2: Shooting targets used to represent bias and variance

Shooting targets, as seen in Figure 2, are not a visual chart used in machine learning, but it does help to explain bias and variance. Imagine that the center of the target, or the bull's-eye, perfectly predicts the correct value of your model. The dots marked on the target then represent an individual realization of your model based on your training data. In certain cases, the dots will be densely positioned close to the bull's-eye, ensuring that predictions made by the model are close to the actual data. In other cases, the training data will be scattered across the target. The more the dots deviate from the bull's-eye, the higher the bias and the less accurate the model will be in its overall predictive ability.

In the first target, we can see an example of low bias and low variance. Bias is low because the hits are closely aligned to the center and there is low variance because the hits are densely positioned in one location.

The second target (located on the right of the first row) shows a case of low bias and high variance. Although the hits are not as close to the bull's-eye as the previous example, they are still near to the center and bias is therefore relatively low. However, there is high variance this time because the hits are

spread out from each other.

The third target (located on the left of the second row) represents high bias and low variance and the fourth target (located on the right of the second row) shows high bias and high variance.

Ideally, you want a situation where there is low variance and low bias. In reality, though, there is more often a trade-off between optimal bias and variance. Bias and variance both contribute to error, but it is the prediction error that you want to minimize, not bias or variance specifically.

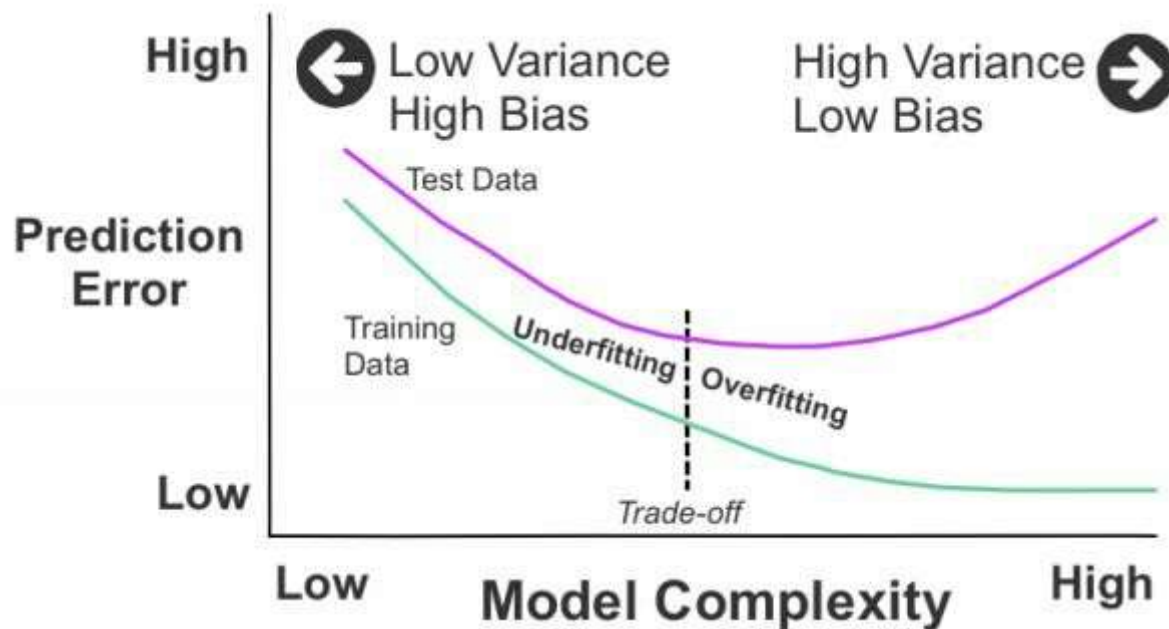


Figure 3: Model complexity based on prediction error

In Figure 3, we can see two lines moving from left to right. The line above represents the test data and the line below represents the training data. From the left, both lines begin at a point of high prediction error due to low variance and high bias. As they move from left to right they change to the opposite: high variance and low bias. This leads to low prediction error in the case of the training data and high prediction error for the test data. In the middle of the chart is an optimal balance of prediction error between the training and test data. This is a common case of bias-variance trade-off.

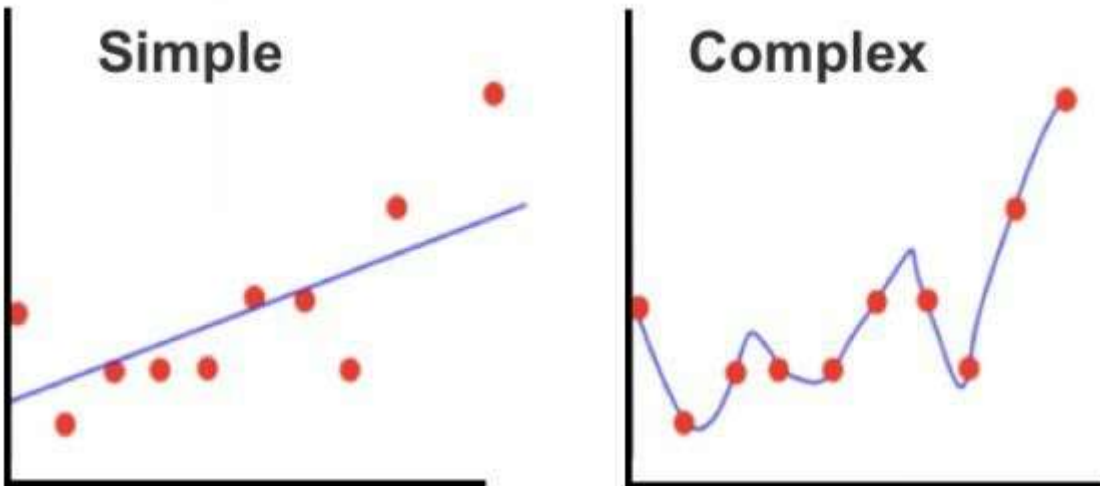


Figure 4: Underfitting on the left and overfitting on the right

Mismanaging the bias-variance trade-off can lead to poor results. As seen in Figure 4, this can result in the model becoming overly simple and inflexible (underfitting) or overly complex and flexible (overfitting).

Underfitting (low variance, high bias) on the left and overfitting (high variance, low bias) on the right are shown in these two scatterplots. A natural temptation is to add complexity to the model (as shown on the right) in order to improve accuracy, but which can, in turn, lead to overfitting. An overfitted model will yield accurate predictions from the training data but prove less accurate at formulating predictions from the test data. Overfitting can also occur if the training and test data aren't randomized before they are split and patterns in the data aren't distributed across the two segments of data.

Underfitting is when your model is overly simple, and again, has not scratched the surface of the underlying patterns in the dataset. Underfitting can lead to inaccurate predictions for both the training data and test data. Common causes of underfitting include insufficient training data to adequately cover all possible combinations, and situations where the training and test data were not properly randomized.

To eradicate both underfitting and overfitting, you may need to modify the model's hyperparameters to ensure that they fit patterns in both the training and test data and not just one-half of the data. A suitable fit should acknowledge major trends in the data and play down or even omit minor variations. This may also mean re-randomizing the training and test data or adding new data points so as to better detect underlying patterns. However, in most instances, you will probably need to consider switching algorithms or modifying your hyperparameters based on trial and error to minimize and

manage the issue of bias-variance trade-off.

Specifically, this might entail switching from linear regression to non-linear regression to reduce bias by increasing variance. Or it could mean increasing “ k ” in k -NN to reduce variance (by averaging together more neighbors). A third example could be reducing variance by switching from a single decision tree (which is prone to overfitting) to a random forest with many decision trees.

Another effective strategy to combat overfitting and underfitting is to introduce *regularization*. Regularization artificially amplifies bias error by penalizing an increase in a model’s complexity. In effect, this add-on parameter provides a warning alert to keep high variance in check while the original parameters are being optimized.

Another effective technique to contain overfitting and underfitting in your model is to perform cross validation, as covered earlier in Chapter 6, to minimize any discrepancies between the training data and the test data.